

# CSE234 Week 9: Copying Algorithms

This week, we will continue discussing the STL algorithms library. This week, we'll discuss the algorithms for copying containers.

## Copying

`std::copy` takes the start and end iterators of the collection being copied, along with the start iterator of the destination collection.

`std::copy_if` takes an additional parameter, a predicate, that should return true if the element should be copied and false if it should not.

`std::copy_n` takes the starting iterator of the collection being copied from, the number of items to copy, and the starting iterator of the destination collection. `std::copy_backward` takes the start and end iterator of the collection to be copied, and the end iterator of the destination collection. The elements are copied in order, but placed at the end of the destination collection.

```
#include <algorithm>
#include <vector>

int main() {
    std::vector<int> values = { 1, 2, 3, 4, 5, 6, 7, 8 };
    // the destination needs to be large enough to hold the result
    std::vector<int> valuesCopy(values.size());
    std::vector<int> evenValues(values.size());
    std::vector<int> firstHalf(values.size() / 2);
    std::vector<int> twoCopies(values.size() * 2);
    // pass the beginning and end of the range to be copied,
    // and the start iterator of the destination to copy it to
    // valuesCopy == { 1, 2, 3, 4, 5, 6, 7, 8 }
    std::copy(values.begin(), values.end(), valuesCopy.begin());
    // pass the beginning and end of the range to be copied from,
    // an iterator to the start of the destination range,
    // and a predicate that returns true if the element should
    // be copied, and false if it should not be copied
    // evenValues == { 2, 4, 6, 8, ?, ?, ?, ? }
    std::copy_if(values.begin(), values.end(), evenValues.begin(), [](
```

```

        return !(x % 2);
    });
    // pass the beginning iterator of the range to be copied from,
    // the number of elements to copy, and the start iterator of
    // the range to copy the elements to
    // firstHalf == { 1, 2, 3, 4 }
    std::copy_n(values.begin(), values.size() / 2, firstHalf.begin());
    // copy values into first half of twoCopies
    std::copy(values.begin(), values.end(), twoCopies.begin());
    // now, use copy_backward to put second copy of values at the end
    // twoCopies == { 1, 2, 3, 4, 5, 6, 7, 8, 1, 2, 3, 4, 5, 6, 7, 8 }
    std::copy_backward(values.begin(), values.end(), twoCopies.end());
    return 0;
}

```

We'll talk about `std::remove` next week, but the functions `std::remove_copy` and `std::remove_copy_if` are discussed here, since they are still copying a collection rather than directly modifying it. These functions will copy a collection to a new collection, but omit elements that meet the specified criteria. Both have the same first three arguments as `std::copy`, the start and end iterator of the collection being copied and the start iterator of the target collection. The fourth argument of `std::remove_copy` is the value that will be omitted when performing the copy. The fourth argument of `std::remove_copy_if` is a predicate that returns true if an element will be omitted, and false if it will be copied.

```

#include <algorithm>
#include <vector>

int main() {
    std::vector<int> values = { 0, 1, 3, -6, 1, 0, -5, 6, -2, 0 };
    std::vector<int> noZeros(values.size());
    std::vector<int> noNegativeValues(values.size());
    // copy values into noZeros, excluding elements that are 0
    // after operation:
    // values == { 0, 1, 3, -6, 1, 0, -5, 6, -2, 0 }
    // noZeros == { 1, 3, -6, 1, -5, 6, -2, ?, ?, ? }
    std::remove_copy(values.begin(), values.end(), noZeros.begin(), 0)
    // copy values into noNegativeValues, excluding elements that are
    // after operation:
    // values = { 0, 1, 3, -6, 1, 0, -5, 6, -2, 0 }
    // noNegativeValues = { 0, 1, 3, 1, 0, 6, 0, ?, ?, ? }
    std::remove_copy_if(values.begin(), values.end(), noNegativeValues
        // predicate returns true if element is ignored
        return x < 0;
    );
}

```

```
});  
return 0;  
}
```

There are also `std::replace_copy` and `std::replace_copy_if` which work exactly the same as the remove variants, but rather than omitting the element they instead take an additional fifth argument, which is a value to replace the target with. See the later section on `std::replace` and `std::replace_if` for an example of replacement.

## Conclusion

This week we discussed the copying algorithms provided in C++. Definitely a shorter week. This week and next week will have a combined lab.